

ASSIGNMENT 5 REPORT - ADVANCED MULTI-CLIENT CHAT SERVER USING TCP

1. OBJECTIVE

Enhance Assignment 4 chat server with advanced client management, private messaging, online-user listing, persistent chat history, and performance evaluation for 2, 3, and 4 clients.

2. SYSTEM ARCHITECTURE

- * Server: 'server.py' on 'h1' (port '5000')
- * Clients: 'client.py' on 'h2' to 'h5'
- * Transport: TCP sockets
- * Shared artifacts: 'chat_history.csv', 'performance_results.csv', 'graphs/', 'screenshots/'

3. DESIGN IMPROVEMENTS OVER ASSIGNMENT 4

- * Added private messaging command: '/msg <username> <message>'
- * Added '/list' command for live online user list
- * Added persistent history in 'chat_history.csv'
- * Added reconnect history playback (last 5 sent messages)
- * Added server-side client state tracking (username/ip/port/login/status)
- * Added server statistics tracking ('server_stats.txt')

4. PROGRAM DESIGN

- * 'server.py' maintains active client state map and routes broadcast/private traffic.
- * 'client.py' supports interactive and auto traffic generation modes.
- * All message events are stored in 'chat_history.csv':
'timestamp,sender,receiver,message_type,message'.

5. EXPERIMENTAL SETUP

- * Mininet topology: 'sudo mn --topo single,5'
- * Mapping: 'h1' server, 'h2..h5' clients
- * Experiments: 2, 3, and 4 clients; each sends 50 messages
- * Capture filter: 'tcp.port == 5000'

6. RESULTS

From 'performance_results.csv':

```
clients | broadcast_messages | private_messages | avg_delay_ms | throughput_msgs_per_sec
---: | ---: | ---: | ---: | ---:
2 | 80 | 20 | 322.531 | 37.945
3 | 120 | 30 | 294.053 | 57.518
4 | 160 | 40 | 480.737 | 75.673
```

7. GRAPH ANALYSIS

Generated graph files:

- * 'graphs/clients_vs_delay.png'
- * 'graphs/clients_vs_throughput.png'
- * 'graphs/message_type_distribution.png'

Observations:

- * Throughput increases with more active clients due higher aggregate message rate.
- * Average delay increases under higher concurrency.
- * Private message counts rise according to configured '/msg' frequency.

8. WIRESHARK VERIFICATION

Filter used: 'tcp.port == 5000'

Screenshots included:

- * 'screenshots/client_connection.png'
- * 'screenshots/broadcast_message.png'
- * 'screenshots/private_message.png'
- * 'screenshots/client_disconnect.png'
- * 'screenshots/tcp_connection_termination.png'

9. REFLECTION ANSWERS

1. Server-side routing is required for private messaging so only the intended recipient receives the message while other clients are excluded.
2. Maintaining user state allows correct online list updates, reconnect behavior, and reliable routing decisions.
3. For 100 users, scale using async I/O or worker pools, optimize shared-state locking, and consider horizontal partitioning.
4. Wireshark packets validating private messaging are client->server payloads

followed by server->single-client payloads instead of broadcast fanout.

5. Next improvement: authentication, encrypted transport, and durable storage.

10. CONCLUSION

Assignment 5 objectives are implemented by extending Assignment 4 with advanced messaging features, persistent history, performance metrics, and packet-level verification evidence.